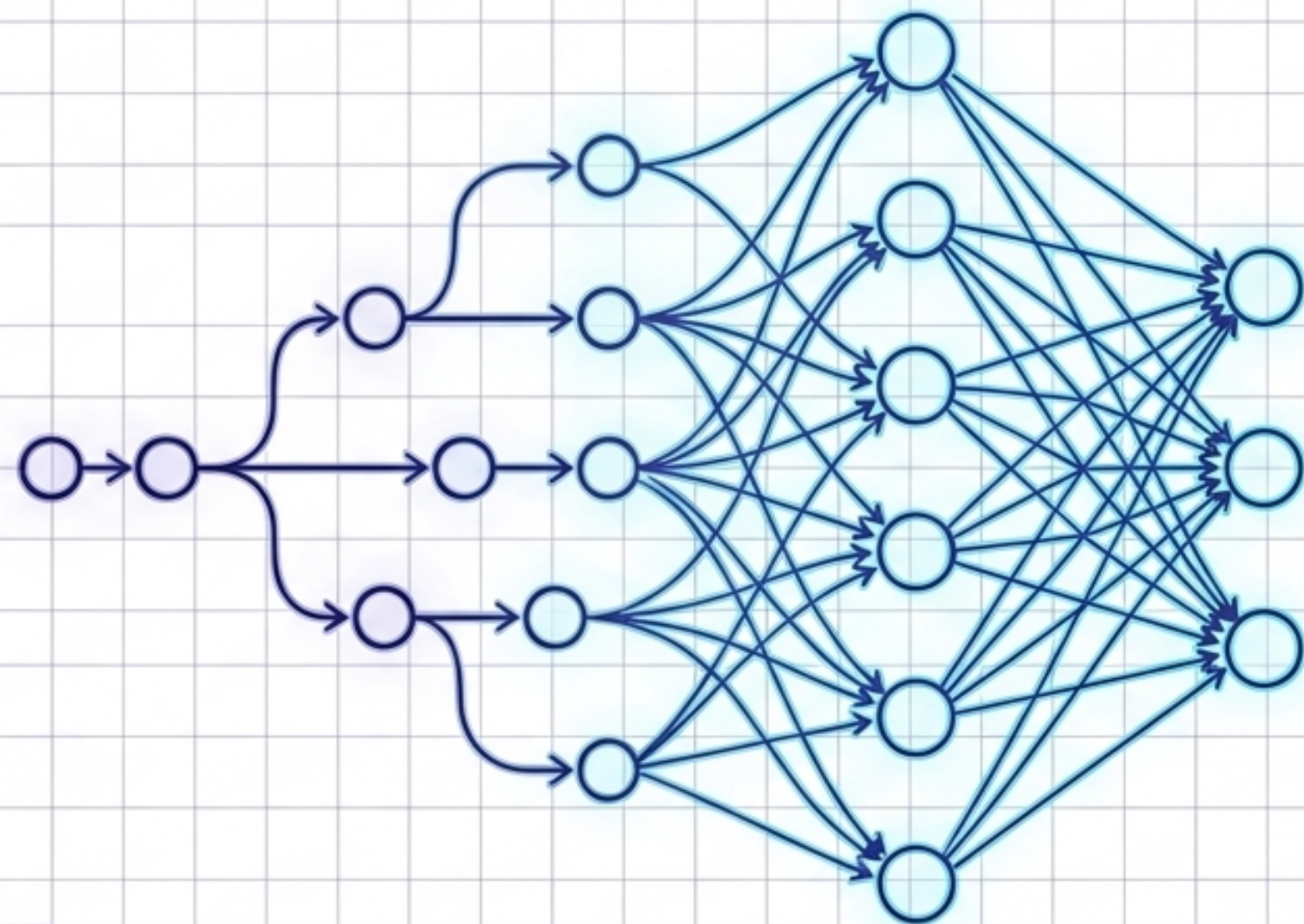


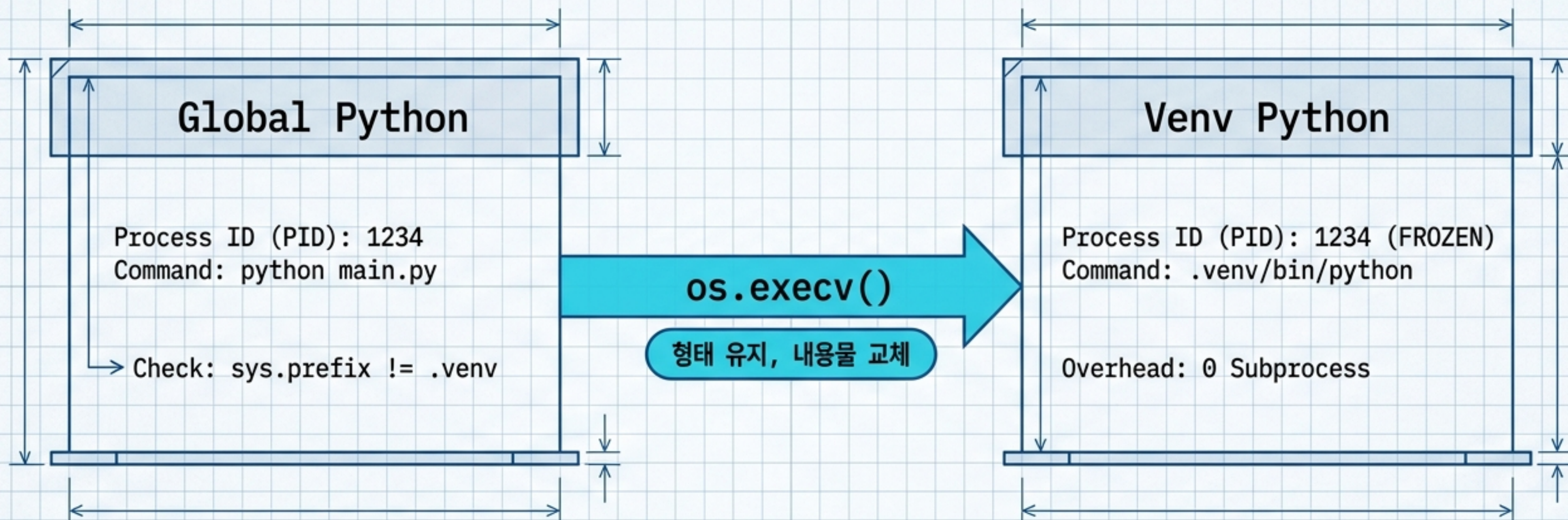
AI Git Assistant: 마이크로 아키텍처 및 실행 파이프라인 심층 분석

확률적 LLM을 결정론적 소프트웨어
시스템으로 통제하는 8단계 엔지니어링
설계도



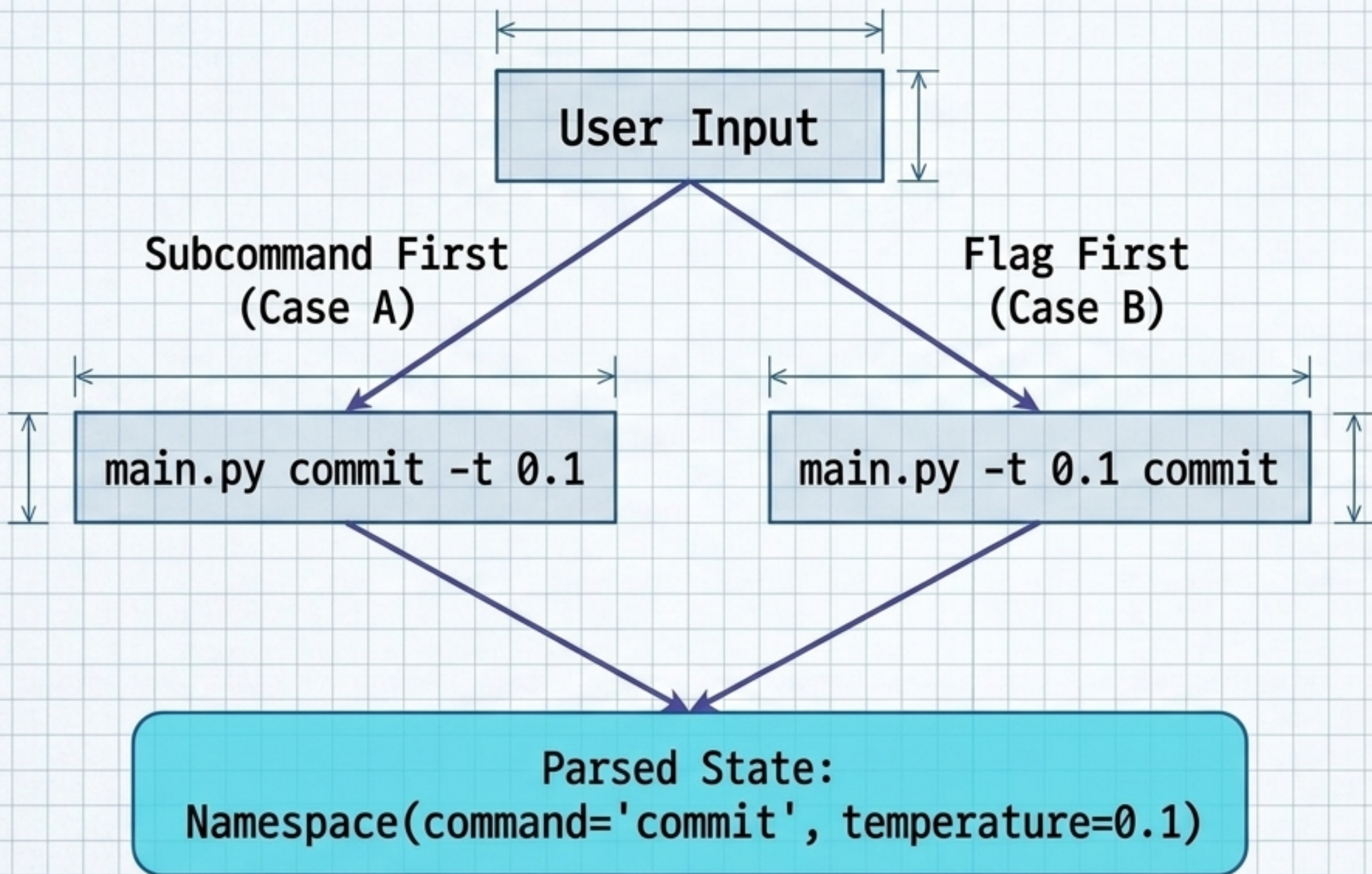
TARGET: Senior Software Engineers & Architects
SCOPE: Source Code Review & Micro-Architecture Analysis
DOCUMENT TYPE: Technical Schematic

01. Zero-Overhead Bootstrapping (os.execv)



서브프로세스(Subprocess) 호출 비용 Zero.
PID(프로세스 ID) 변경 없이 투명하게 가상환경으로 제어권 이관.

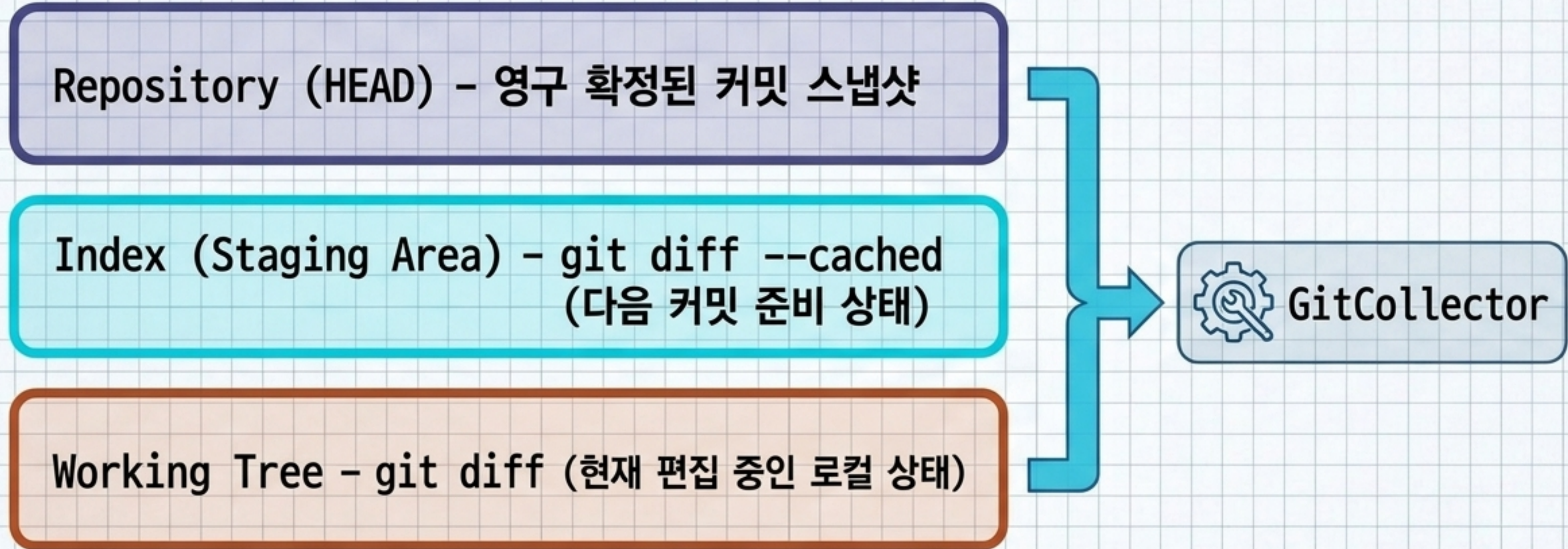
02. Position-Independent CLI 파싱 아키텍처



argparse.SUPPRESS 적용

서브커맨드와 글로벌 플래그
간의 기본값(Default Value)
충돌을 원천 차단하여
위치 자유도 100% 보장.

03. Git Context 수집 매커니즘 (3-Way Tree)

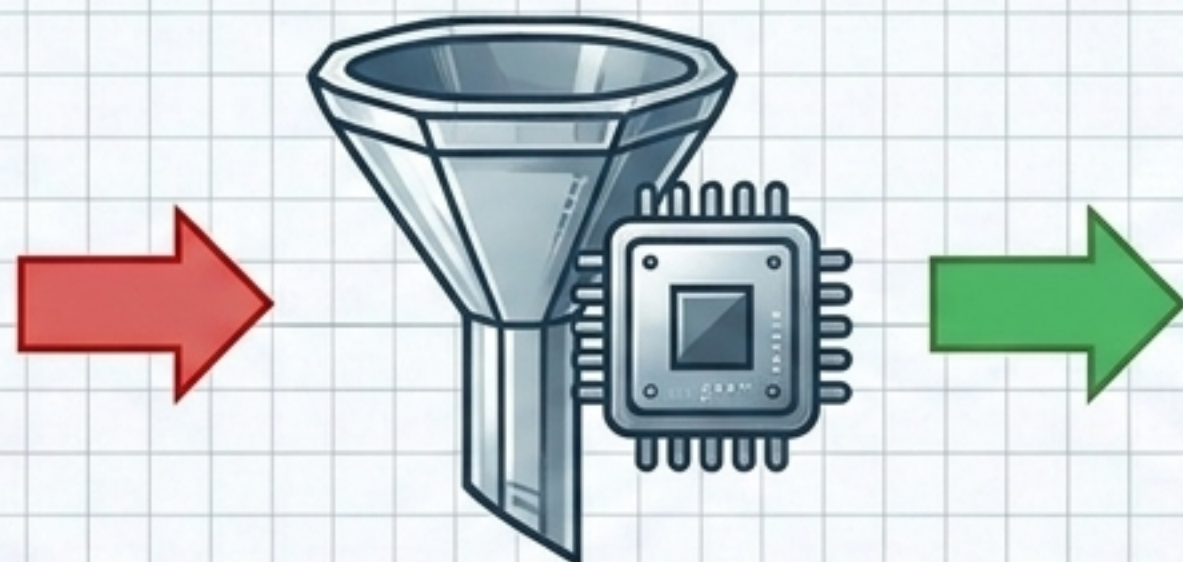


Staged와 Unstaged의 diff를 단일 문자열로 통합 수집하여
변경 사항 누락 방지 및 0건일 경우 API 호출 전 조기 정상 종료(`sys.exit(0)`).

04. Payload Security: 정규식 기반 9종 마스킹

Input (Raw Code)

```
api_key = 'sk-ant-12345ABCD...'  
email: dev@codyssey.kr
```



src/git_collector.py:_mask_sensitive
(9종 정규표현식 검사)

Output (Safe Payload)

```
api_key = '[MASKED_ANT_KEY]'  
email: '[MASKED_EMAIL]'
```

파일 10개, 최대 200줄 초과 시
자동 하드 절삭(Truncation) 알고리즘 적용 (--safe-mode).

05. LLM 파라미터 해부: 전송 비대칭성

Codyyssey Gateway로 전송 (Request)

```
{  
  'model': 'gpt-5.4-mini',  
  'messages': [...],  
  'temperature': 0.3,  
  'max_tokens': 1024  
}
```

서버로부터의 반환 (Response)

```
{  
  'id': 'chatcmpl-7cb6...',  
  'model': 'gpt-5.4-mini',  
  'choices': [{ 'message': {  
    'content': '...',  
    'finish_reason': 'stop'  
  } }],  
  'usage': { 'total_tokens': 18 }  
}
```

하이퍼파라미터(temperature, max_tokens)는 철저히 Request-side 제어 변수이며, LLM 서버의 Response Body에는 Echo-back 되지 않음.

06. 하이퍼파라미터 심층 분석: temperature

docs: 컨벤션 로드와 LLM
파라미터 검증 기록 추가

- 명사형 종결
- 사실적 요약
- 95% 이상 고정 재현율

0.1 (결정론적/엄격)

docs: 컨벤션 로드 동작과 LLM
파라미터 검증 내용을 정리하라

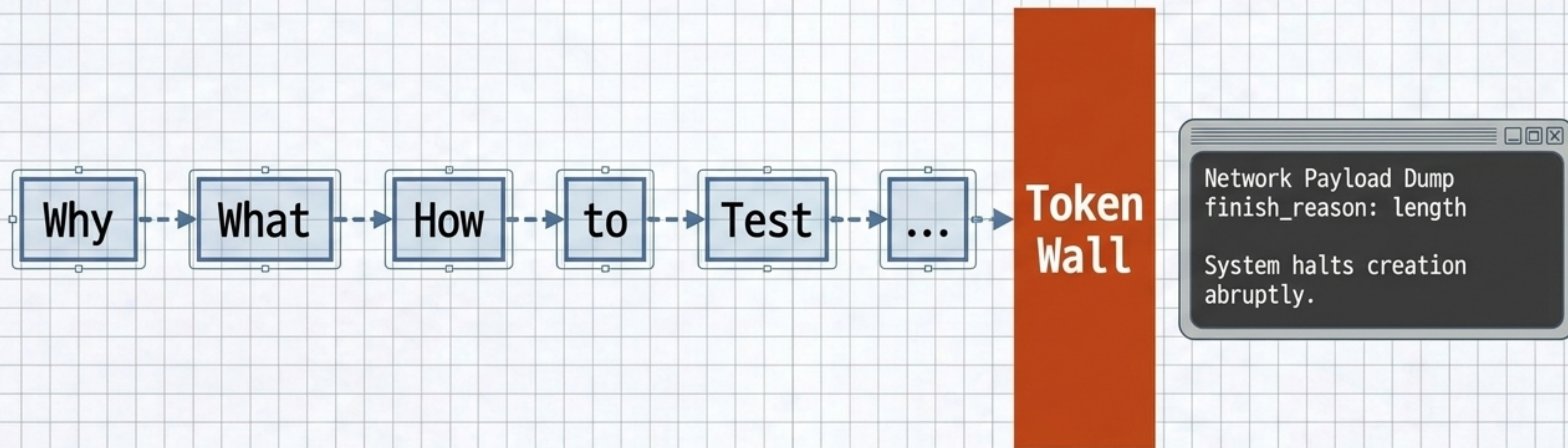
- 명령형 어미
- 풍부한 어휘 샘플링
- 출력 변동성 높음

0.8 (무작위성/다양성)

낮은 온도는 소프트맥스 확률 분포 최상단 토큰(Top Token)을
강제하여 사실 기반의 Git 요약에 최적화됨.

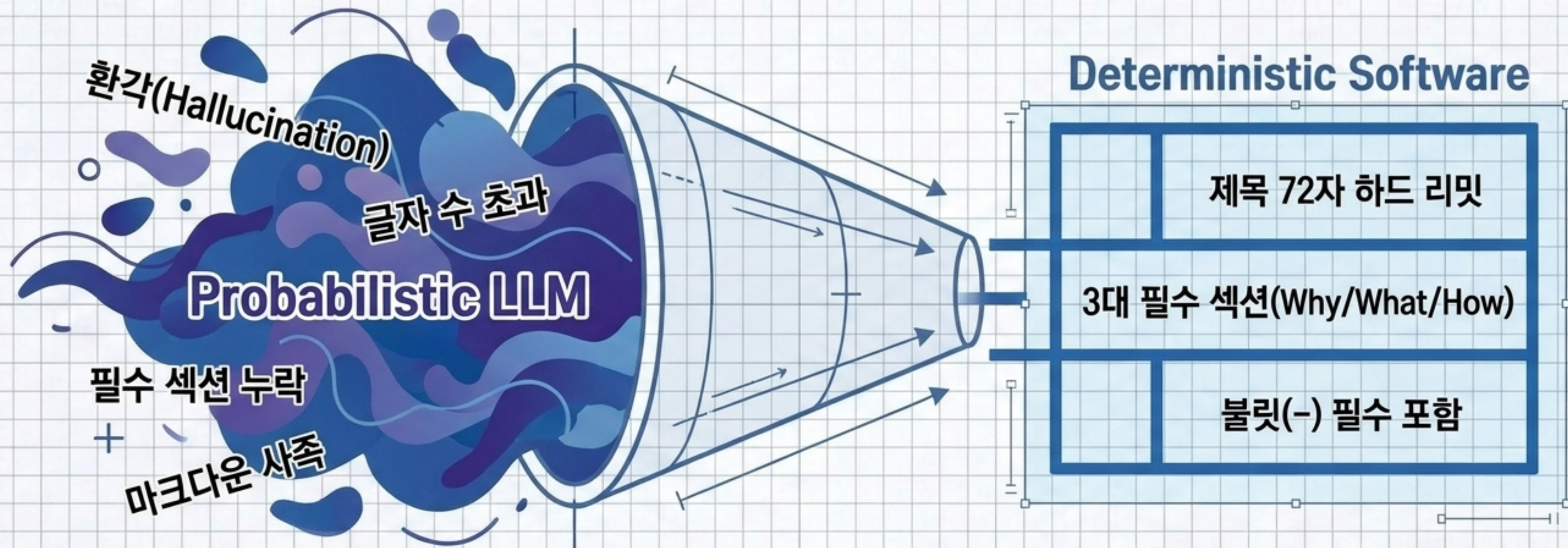
07. 하이퍼파라미터 심층 분석: max_tokens

max_tokens: 20 (Extreme Test Case)



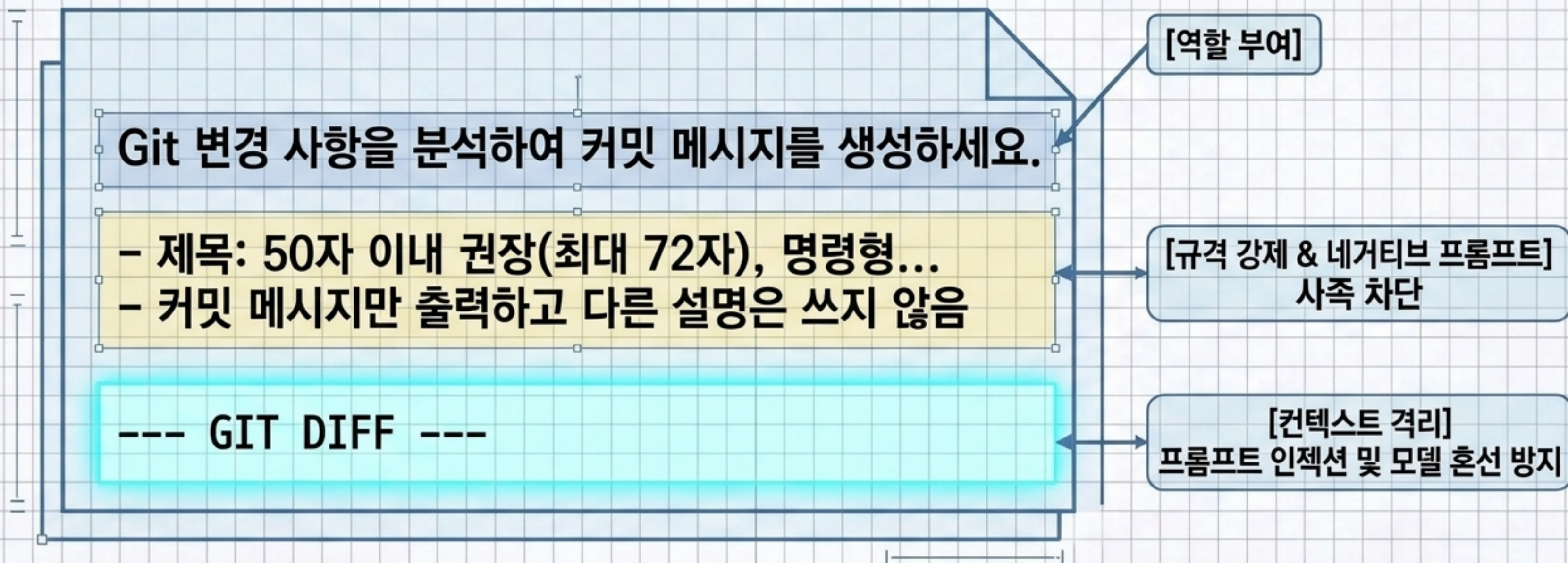
Design Decision: PR 본문(약 600토큰 소모)이 중도 절삭되는 것을 방지하기 위해 기본값을 1024로 넉넉하게 할당하여 finish_reason: stop을 보장.

08. 아키텍처 딜레마: 확률적 모델 vs 결정론적 포맷



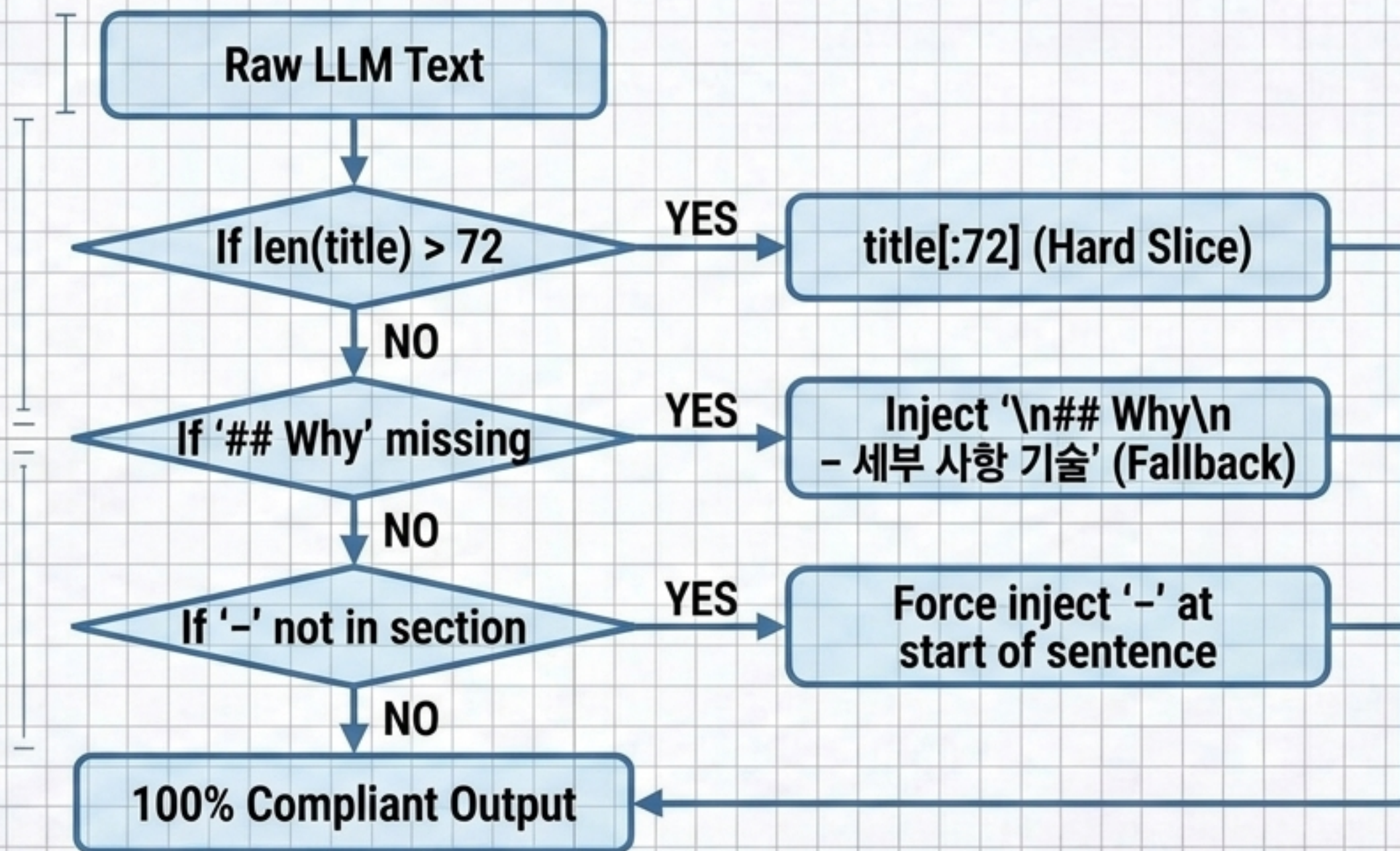
어떻게 확률적 텍스트 생성기를 100% 규격화된 CI/CD 파이프라인의 입력으로 신뢰할 수 있게 만들 것인가?

09. 이중 안전장치 1단계: Inference Guidance (프롬프트 설계)



LLM의 맥락을 90% 이상 올바른 방향으로 좁히는 1차 소프트 가이드레일.

10. 이중 안전장치 2단계: 결정론적 사후 검증 (Validator)



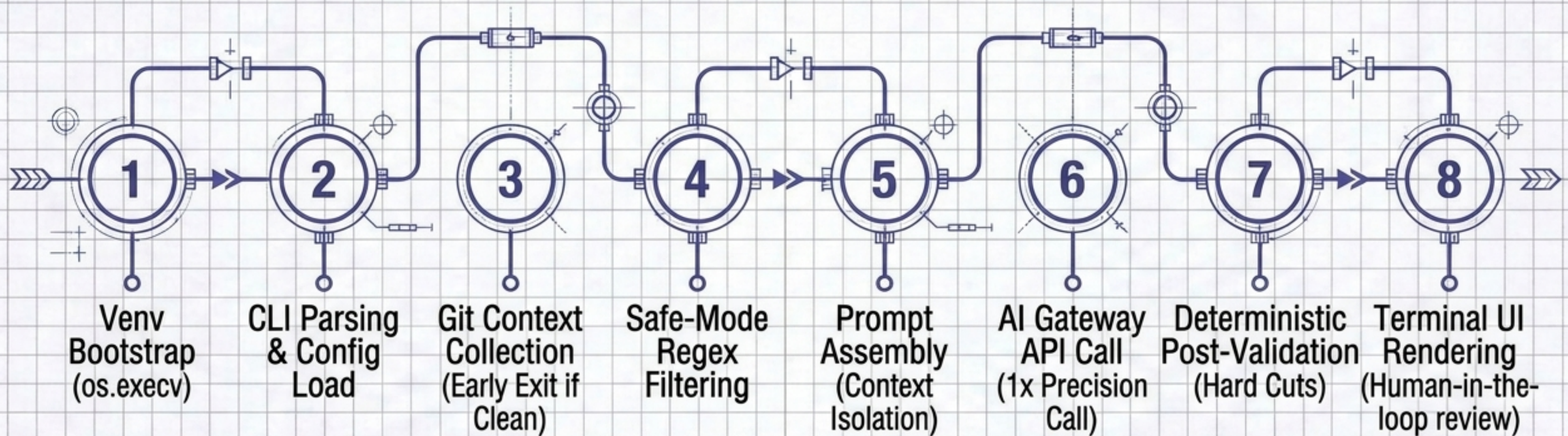
확률적 실패를 0.001초 만에 파이썬 레벨 알고리즘으로 자동 치유(Auto-healing)하여 100% 규격 준수 확정.

11. ADR: 규칙 위반 대응 방안 (재생성 vs 후처리)

평가 기준 (Criteria)	재생성 (Retry)	후처리 (Post-processing) [선택]
모델 (Model)	API 재호출 (프롬프트 수정)	파이썬 정규식 및 슬라이싱
응답 지연 (Latency)	~4.0초 (UX 저하)	~1.5초 (초고속 유지)
규칙 준수 보장성 (Guarantee)	확률적 (보장 불가)	수학적 100% 확정
토큰 비용 (Cost)	2배 증가	추가 비용 없음 (1회 정밀 호출 고수)

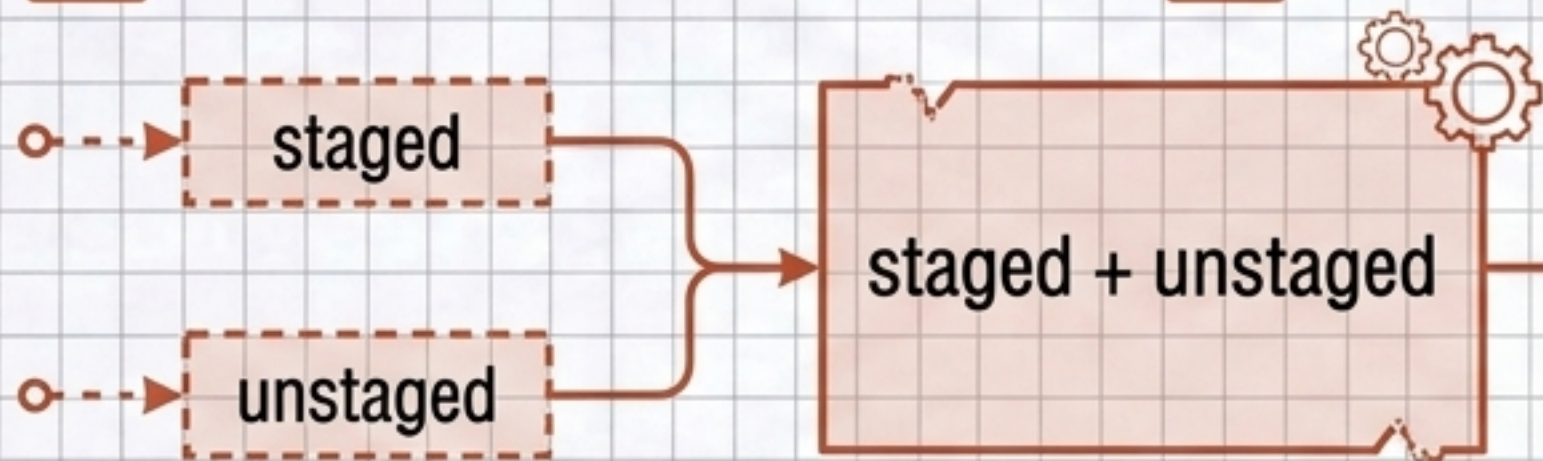
규칙 위반 시, 느리고 비싼 ‘재생성’ 대신 빠르고 확실한 ‘후처리’를 선택하여 시스템 안정성과 효율성을 극대화함.

12. Synthesis: 8-Step Execution Pipeline

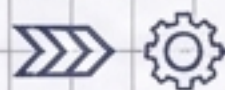


13. 한계점 분석 및 차세대 아키텍처 제안

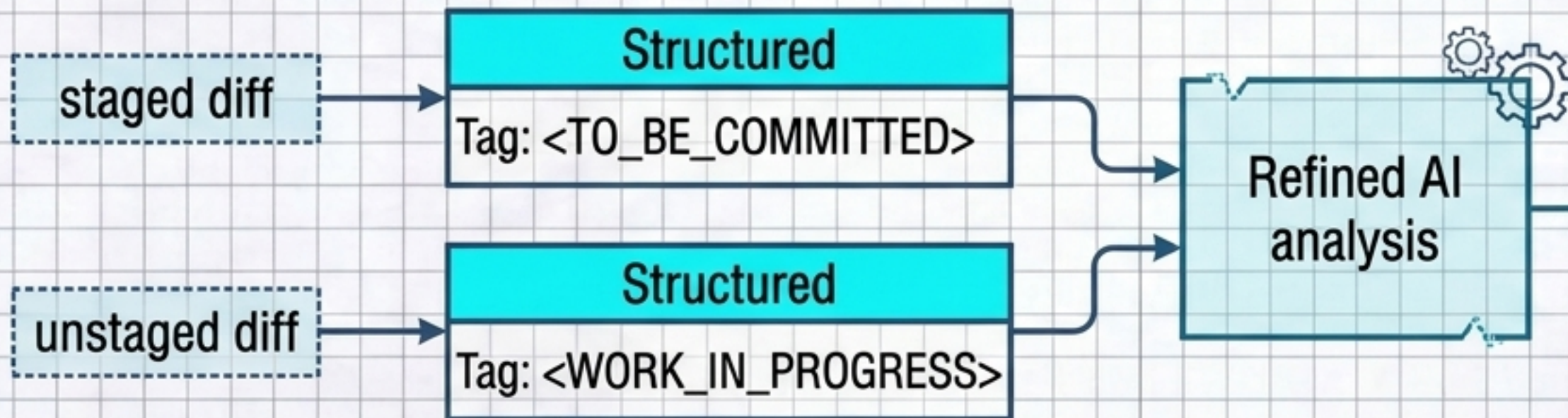
! CURRENT ARCHITECTURE !



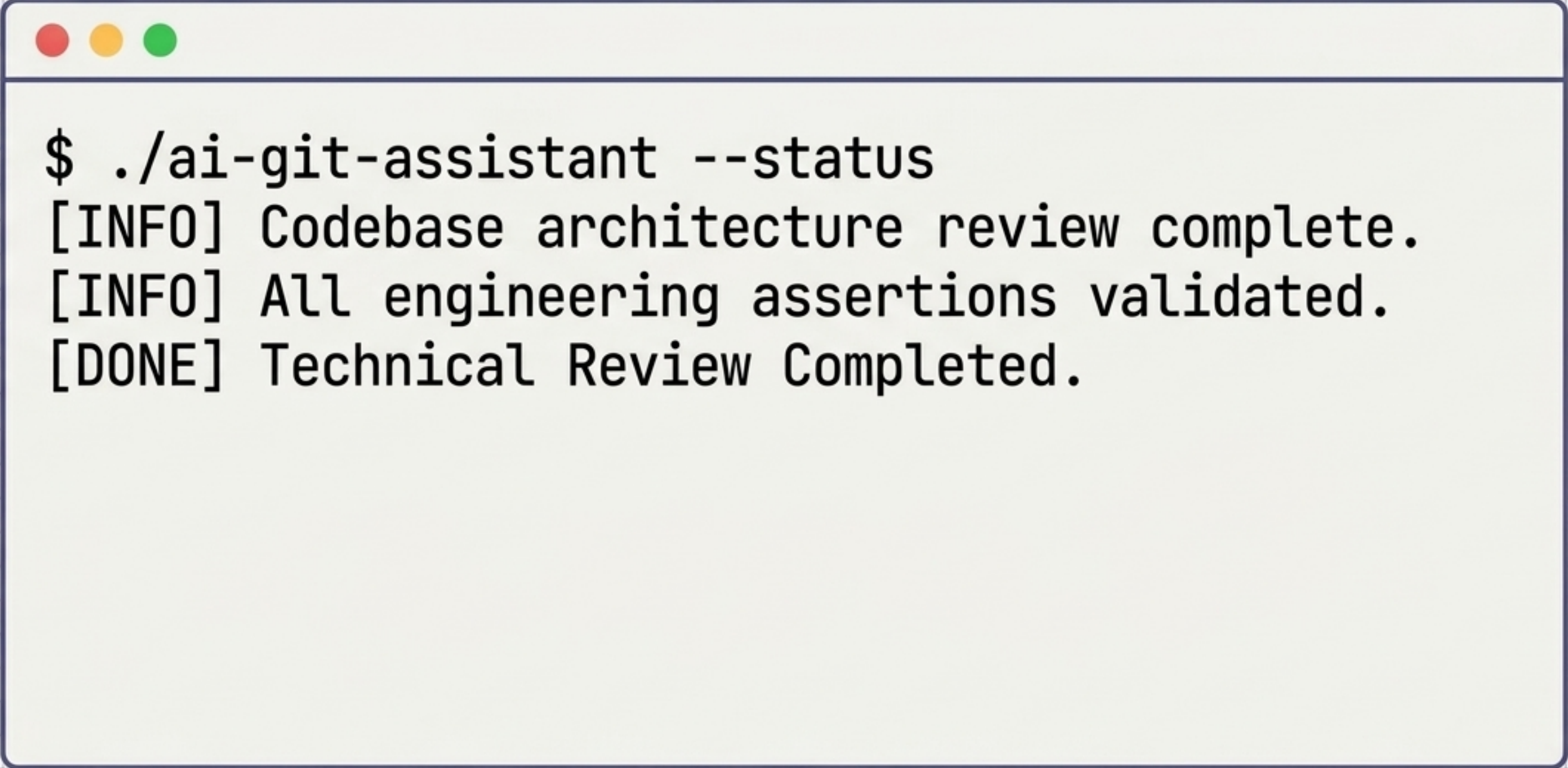
단순 문자열 결합으로 인해 AI가 커밋 대상(Index)과 작업 중인 코드(Working Tree)를 명확히 구분하지 못하는 할루시네이션 리스크 존재.



🔒 FUTURE ARCHITECTURE: Staged-First Priority 🔒



상태 구분 헤더 주입 및 Staged 변경점 최우선 분석 정책 도입 예정.



```
$ ./ai-git-assistant --status  
[INFO] Codebase architecture review complete.  
[INFO] All engineering assertions validated.  
[DONE] Technical Review Completed.
```

GitHub Repository: [nttkor/b3_2](#)